📦 교양으로 알아가는 Rootless Container

문지현

About me

문지현

- 5년차 엔지니어
- Kubernetes & Cloud 기반으로 서비스 인프라를 고도화하고, 개선하는 일을 하고 있습니다.
  - CNKCD 2025: Metric & Trace 제대로 챙기기: Grafana Stack + Alloy 중심의 데이터 수집/저장 효율화 과정
  - CNKCD 2024: 선언형으로 만드는 멀티 클러스터 - Cluster API와 App of Apps 패턴

Career

- **Kbank (2024.08 ~ now)**, SRE
- **Smilegate Megaport (2022.01 ~ 2024.07)**, Platform Engineering & Cloud Tech

질문:

서버에 Root 권한 없이 컨테이너나 K8S 올릴 수 있나요?

~~답변:~~

~~음... 웬만하면 Root 쓰는게 더 편할텐데...~~

Rootless Container의 탄생 배경

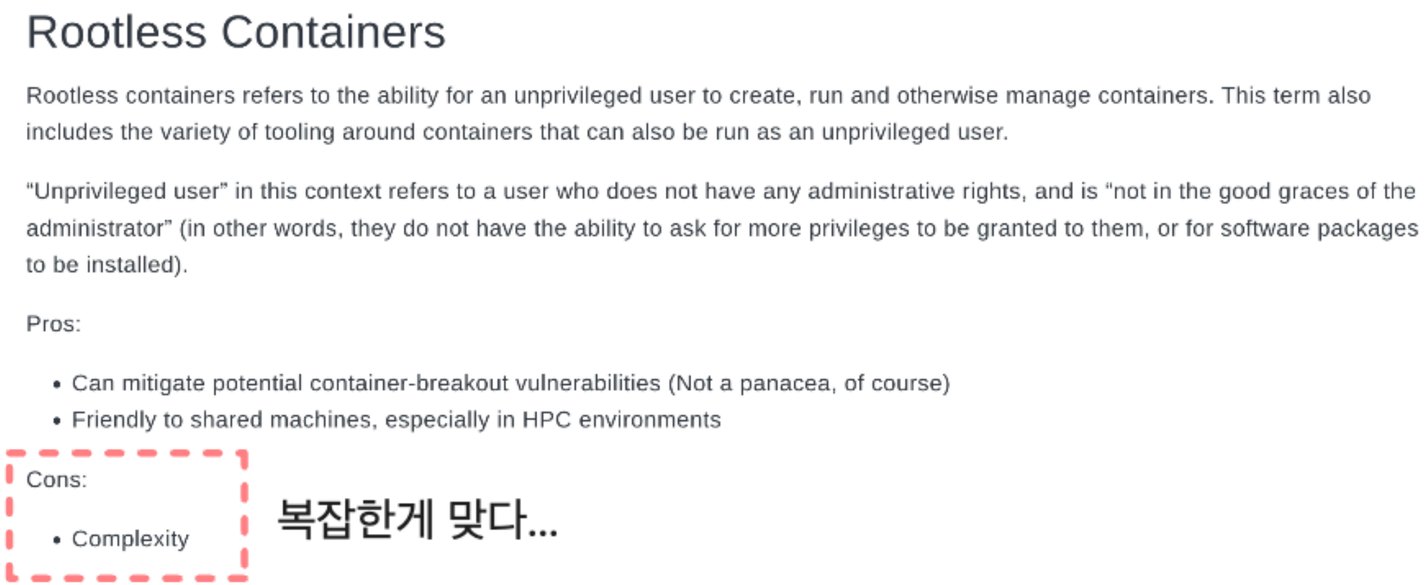
root를 사용할 수 없는 환경에서도 컨테이너를 쓰고싶어요.

소개 사이트: https://rootlesscontaine.rs/

목표: unprivileged 권한으로 컨테이너 및 컨테이너 런타임을 실행할 수 있도록 한다.

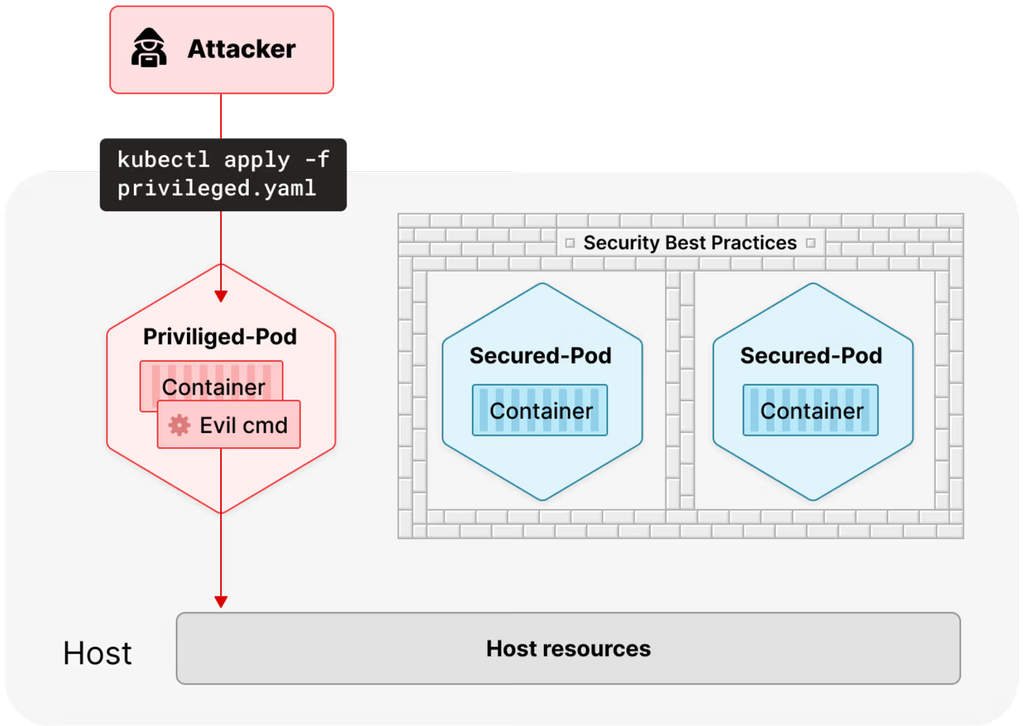
(참고) Rootless Containers with runC

- https://www.cyphar.com/blog/post/20160627-rootless-containers-with-runc
- 서버 root 권한을 절대 받을 수 없는 환경에서도 컨테이너를 쓰고싶다는 니즈에서 시작함.



Rootless Container의 탄생 배경

root를 쓰는 한, 컨테이너 탈출(container escape/breakout) 문제에 걸림.




```
$ cat privileged.yaml

apiVersion: v1
kind: Pod
metadata:
  name: privileged-the-pod
spec:
  hostPID: true
  hostNetwork: true
  containers:
  - name: privileged-the-pod
    image: nginx:latest
    ports:
    - containerPort: 80
    securityContext:
      privileged: true
```

K8S Pod를 통해 일어날 수 있는 컨테이너 탈출(==노드 탈취) 케이스 중 “일부”

출처: https://isovalent.com/blog/post/2021-11-container-escape/

Rootless Container의 탄생 배경

root를 쓰는 한, 컨테이너 탈출(container escape/breakout) 문제에 걸림.

🔨 물론 K8S 레벨이라면 여러가지 방법으로 방지할 수 있을 것이다.

- Validating Webhook 기반으로 규칙 강제화 하거나
- K8S의 기본적인 기능 안에서도 이러한 강제/제어에 대한 내용이 있음. (Pod Security Admission 등)

🔨 다만 컨테이너 생태계에서는, 근본 원인 자체를 해결하려는 시도 또한 이뤄지고 있다.

- KEP-2033: Kubelet-in-UserNS (aka Rootless mode)
- 컨테이너 탈출 취약점을 해결하기 위한 방향으로, non-root에서 구동되는 kubelet이 나오게 됨.
  - https://github.com/kubernetes/enhancements/tree/master/keps/sig-node/2033-kubelet-in-userns-aka-rootless#kep-2033-kubelet-in-userns-aka-rootless-mode

Rootless Container는
어떻게 구성된건지 궁금하지 않나요? 👀

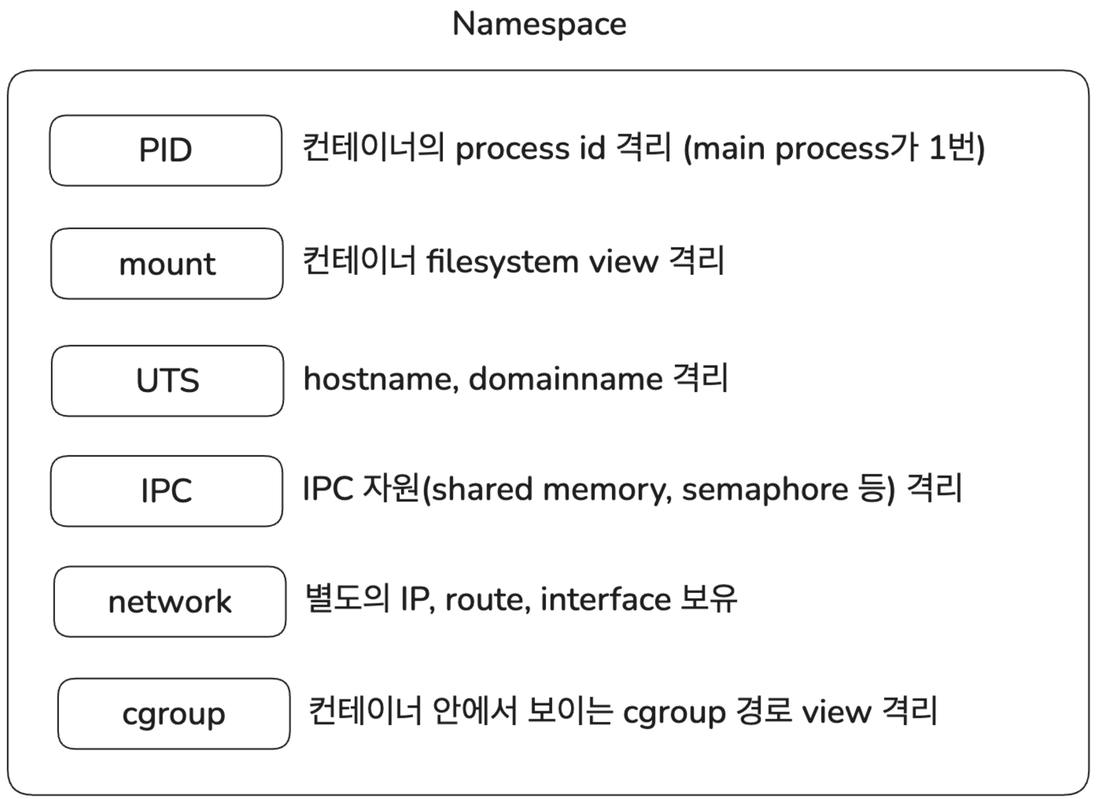
한번 살펴봅시다!

Index

- **Background**
  - **Rootless Container의 탄생 배경**
- **Rootless Container 구조**
  - **user_namespace를 통한 UID/GID 분리**
    - UID/GID 계산법 및 기존 Rootful Container와 비교
  - **rootless 환경에서의 컨테이너 네트워크 구현**
    - 다양한 구현체 중 Slirp에 대한 소개
    - Rootless Container의 Outbound 통신 구조
    - Rootless Container의 Inbound 통신 구조
- **정리 및 결론**

컨테이너는 어떻게 만들어지나요?

컨테이너 자체는 리눅스 Namespace를 통해 만들 수 있음.



[참고] 컨테이너 직접 만들어보기 (한번 쯤 해보면 유익합니다)

- 이게 돼요? 도커 없이 컨테이너 만들기 / if(kakao)dev2022
  - https://youtu.be/lVtgqmjv4BQ?si=vJTu2IVb_KvpiMYW

그런데, non-root 권한으로 안되는게 있음.

namespace 외에도 다른 여러가지 세팅들도 필요.

```
# host NAT로 인터넷 연결 & Port Forwarding 설정 예시
$ nft list ruleset
table ip mini_nat {
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;
        ip saddr 10.200.1.0/24 oifname "ens5" masquerade
        ip saddr 127.0.0.0/8 ip daddr 10.200.1.2 tcp dport 8080 oifname
"veth-host" snat to 10.200.1.1
    }

    chain prerouting {
        type nat hook prerouting priority dstnat; policy accept;
        tcp dport 18080 dnat to 10.200.1.2:8080
    }

    chain output {
        type nat hook output priority -100; policy accept;
        ip daddr 127.0.0.1 tcp dport 18080 dnat to 10.200.1.2:8080
    }
}
```

veth 기반 컨테이너 네트워크

```
# cgroup controller 설정 예시
CG=/sys/fs/cgroup/mini-c1
mkdir -p "$CG"

# memory.max = 128MiB
echo $((128 * 1024 * 1024)) > "$CG/memory.max"

# cpu.max = 100ms 중 50ms 사용 가능
echo "50000 100000" > "$CG/cpu.max"

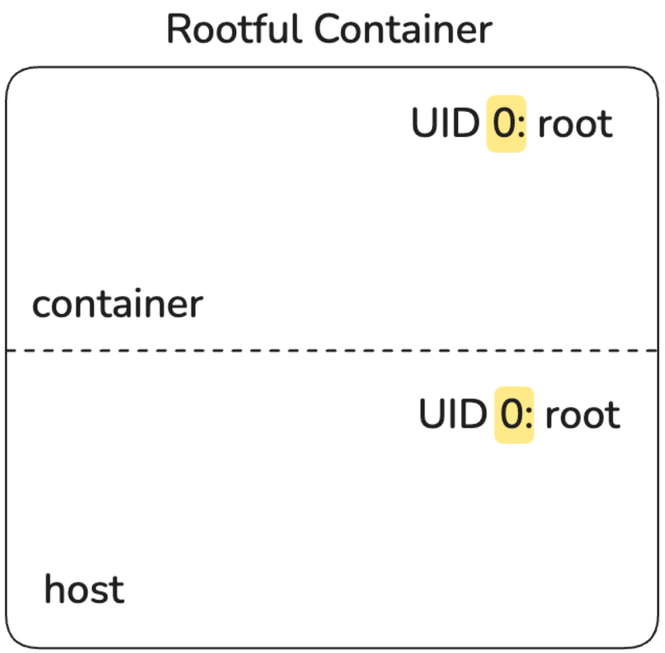
# pids.max = 최대 프로세스 수 64개
echo 64 > "$CG/pids.max
```

cgroup controller 설정

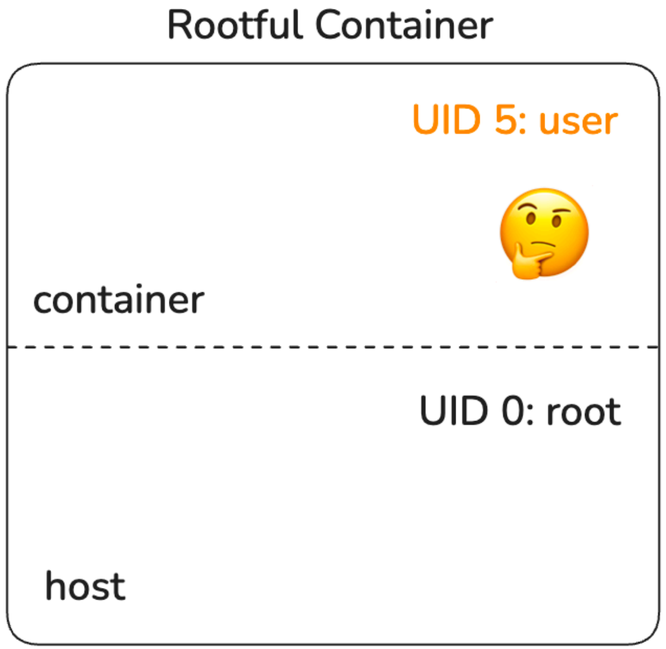
+host 레벨에서 필요한 설정 변경 (sysctl 등) 및 패키지 설치 등등…

root를 쓸 수 없는 상황인데 root 자체는 필요함.

기존 Rootful Container 구조에서 맞춰보려고 하니...



기본적으로 container 상의 UID와
host 상의 UID 번호는 동일하다.



컨테이너 이미지에서 root를 빼서 사용?
범용성을 보장하기 어렵다...

user_namespace로 UID/GID를 격리할 수 있다.

이렇게 되면 user_ns 안에서는 마치 내가 root인 것처럼 보임.

```
[user@my-server ~]$ whoami
user
                                    user_namespace 사용!
[user@my-server ~]$ unshare --user --map-root-user \
--mount --uts --pid --fork bash

# !!
# 마치 root user인 것처럼 보이게 된다.
[root@my-server ~]#

[root@my-server ~]# whoami
root
```

user_ns 기반의 Rootless Container

newuidmap/newgidmap으로 컨테이너가 사용할 uid/gid 범위 구현 가능.

```
$ cat /etc/subuid
my-user:100000:65536

$ cat /etc/subgid
my-user:100000:65536
```

1. username 혹은 UID
2. 종속된 UID가 시작되는 번호
3. 종속된 UID의 할당 가능 개수

즉, 이 경우 100000 ~ 165535 범위에서 할당 가능

이게 필요한 이유? ➡️ 하나의 컨테이너에서 UID/GID가 복수로 나올 수 있기 때문.

user_ns 기반의 Rootless Container

이때, host 상에 표시되는 컨테이너 UID는 미리 계산할 수 있다.

```
$ cat /proc/$CONTAINER_PID/uid_map

         0       1001          1
         1     100000      65536
```

1. UID 시작값
2. Host UID 시작값
3. 할당할 UID 범위

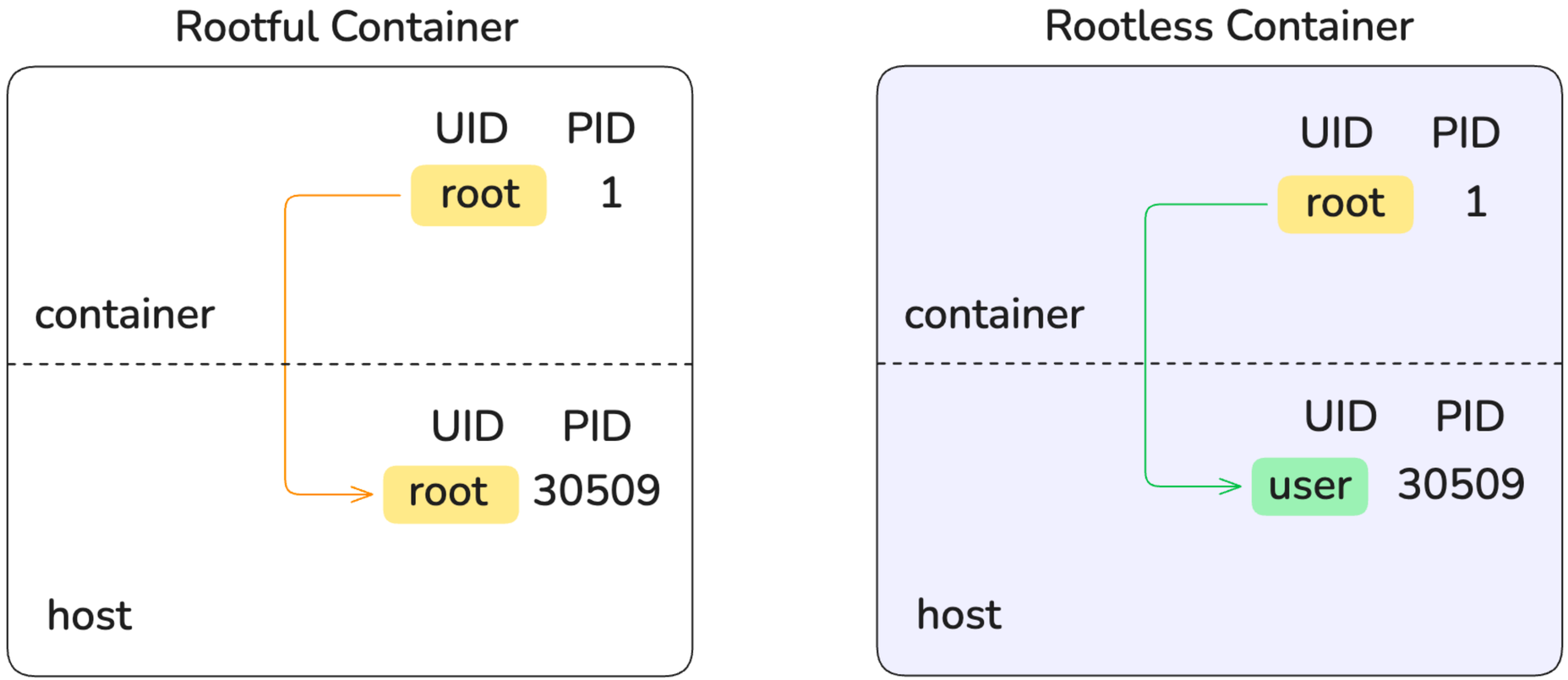
Host 상의 컨테이너 UID =
[2] - [1] + 컨테이너 상의 UID

즉, 1 100000 65536의 경우

컨테이너 상의 UID		Host 상의 UID
1		100000
...	→	...
65536		165535

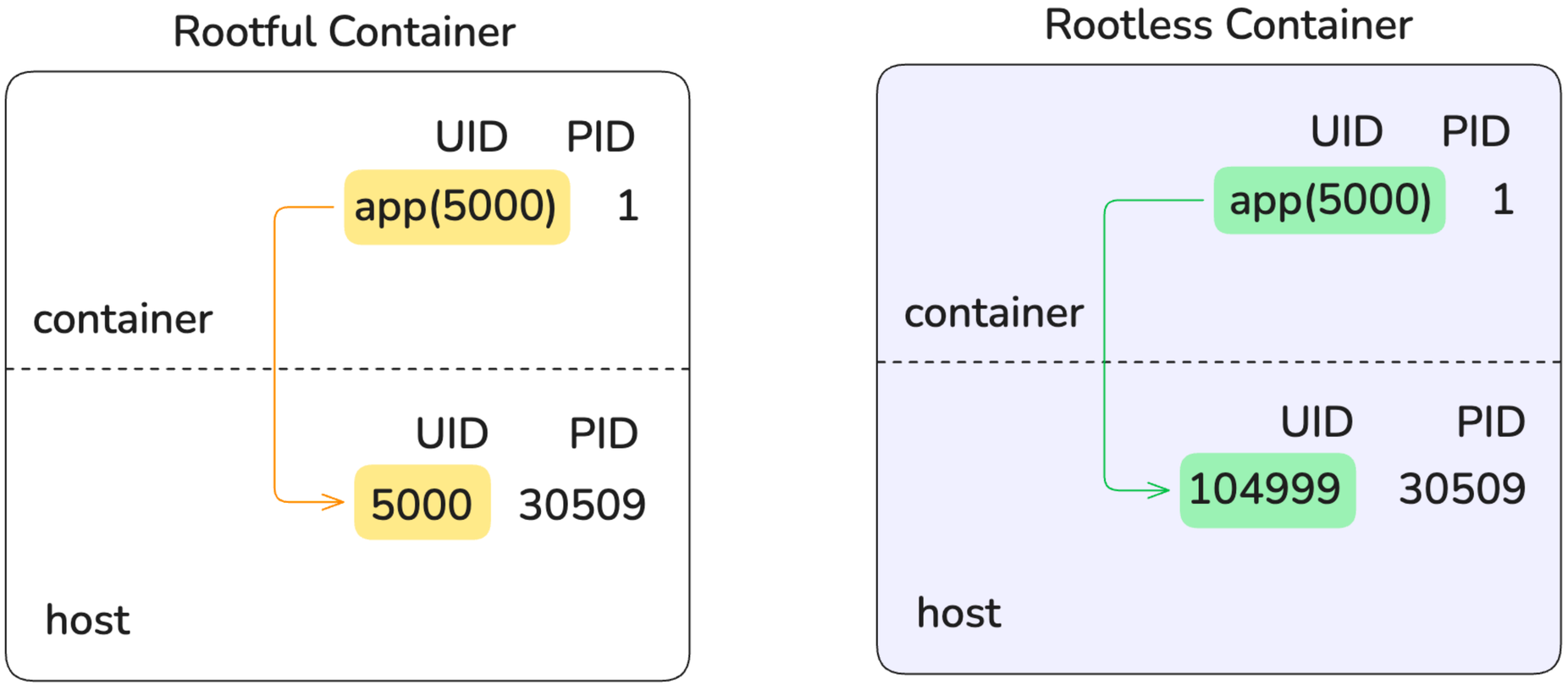
컨테이너 UID 구성 비교 (root user)

1. Rootless는 컨테이너의 root user 프로세스 UID를 user 레벨로 고정한다.



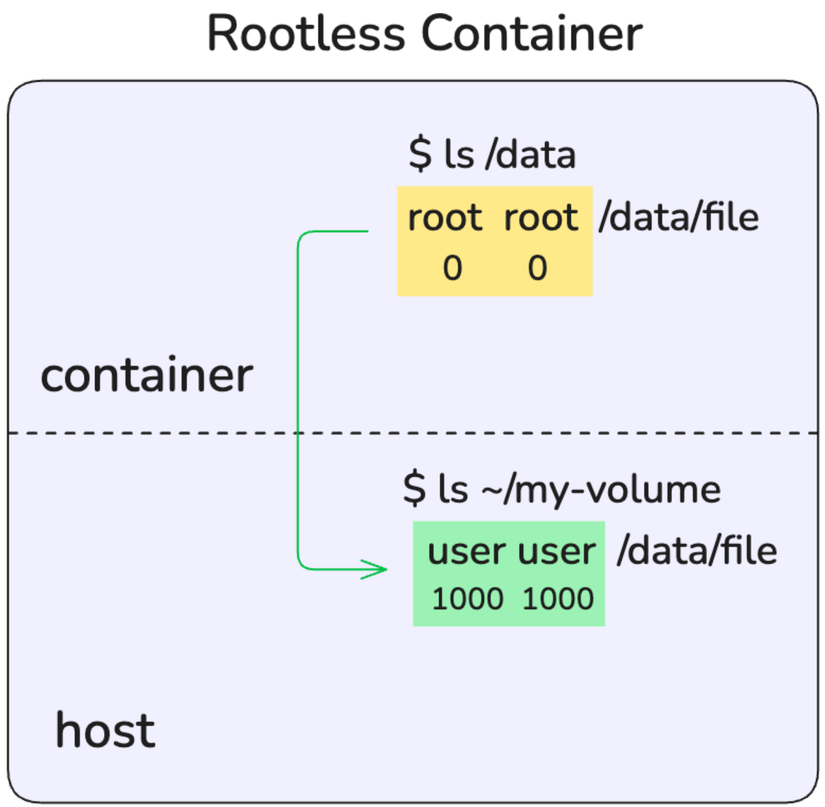
컨테이너 UID 구성 비교 (non-root user)

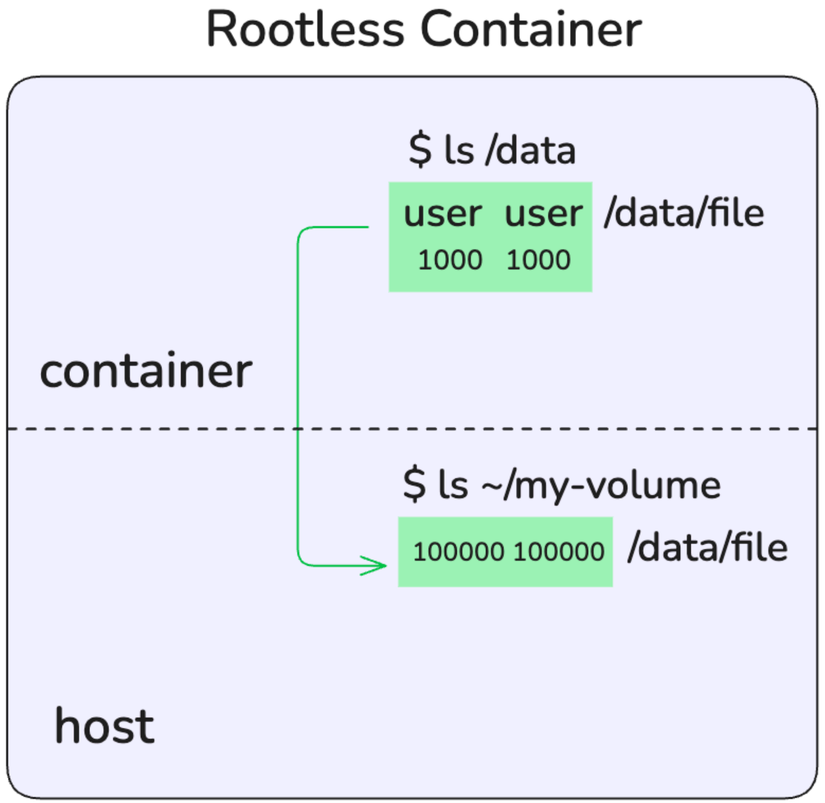
2. Rootless는 컨테이너의 non-root user 프로세스 UID를 user 레벨로 고정한다.



컨테이너 UID 구성 비교

3. Rootless의 파일 소유 권한 uid/gid 구성도 앞의 내용과 동일하게 적용된다.

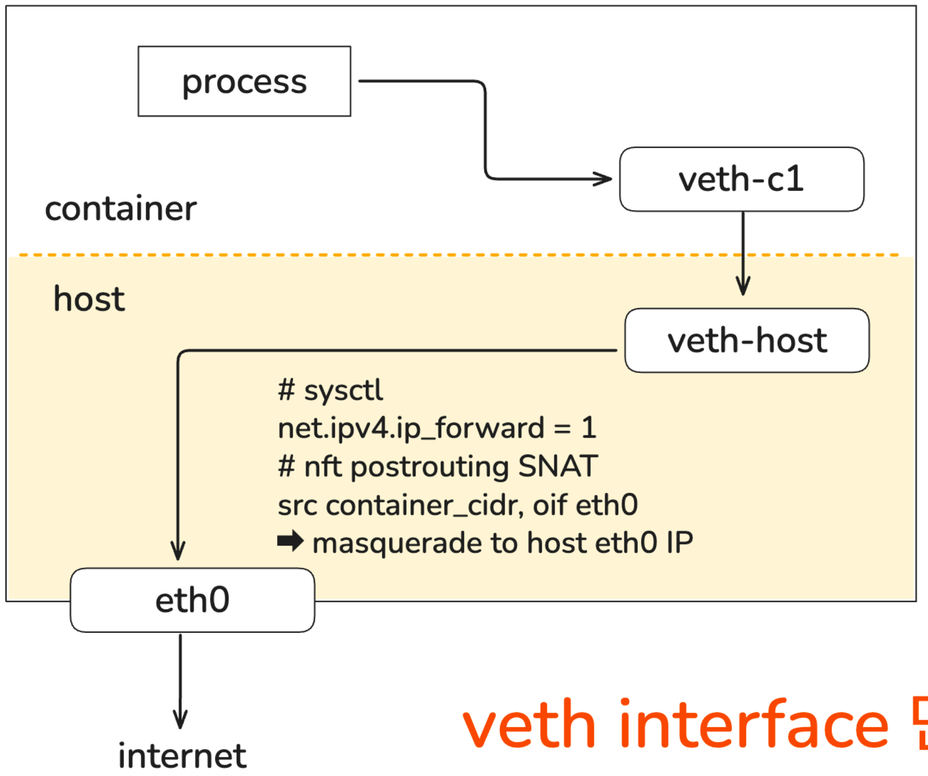




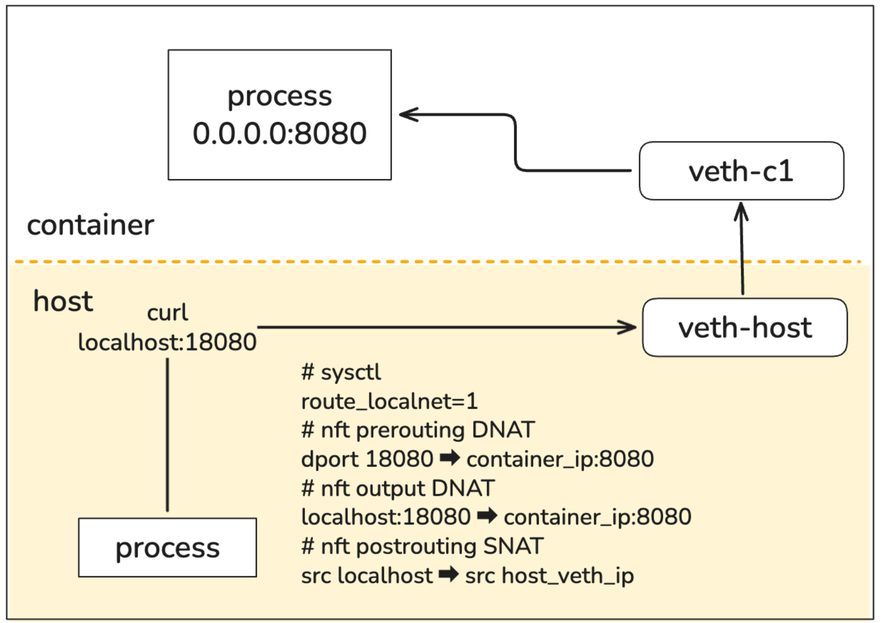
컨테이너 네트워크는 Rootless에서 까다로움.

일반적인 구성을 따라가려니, host root 레벨의 권한이 필요하다.

host를 통한 Internet 통신 예시



컨테이너 port-forward 구현 예시



veth interface 만들 때 CAP_NET_ADMIN 필요

이를 해결할 목적의 다양한 네트워크 구현체가 존재

Rootless에서는 Outbound / Inbound 통신 각각 상황이 다르기 때문.

- slirp4netns (+RootlessKit)
- pasta
- VPNKit
- bypass4netns
- gvisor-tap-vsock
- 기타 등등…

이를 해결할 목적의 다양한 네트워크 구현체가 존재

Rootless에서는 Outbound / Inbound 통신 각각 상황이 다르기 때문.

- slirp4netns (+RootlessKit) 🤔
  - 이걸로 Outbound / Inbound 통신을 파악해봅시다.
- pasta
- VPNKit
- bypass4netns
- gvisor-tap-vsock
- 기타 등등…

배경 지식: Slirp가 무엇인가요?

알고보니, 지금으로부터 30년 전에 나온 구현체.

인터넷 사용이 어렵던 시절, 남의 PC의 non-root를 경유해 내 PC에서 인터넷을 쓰기 위해 탄생함.

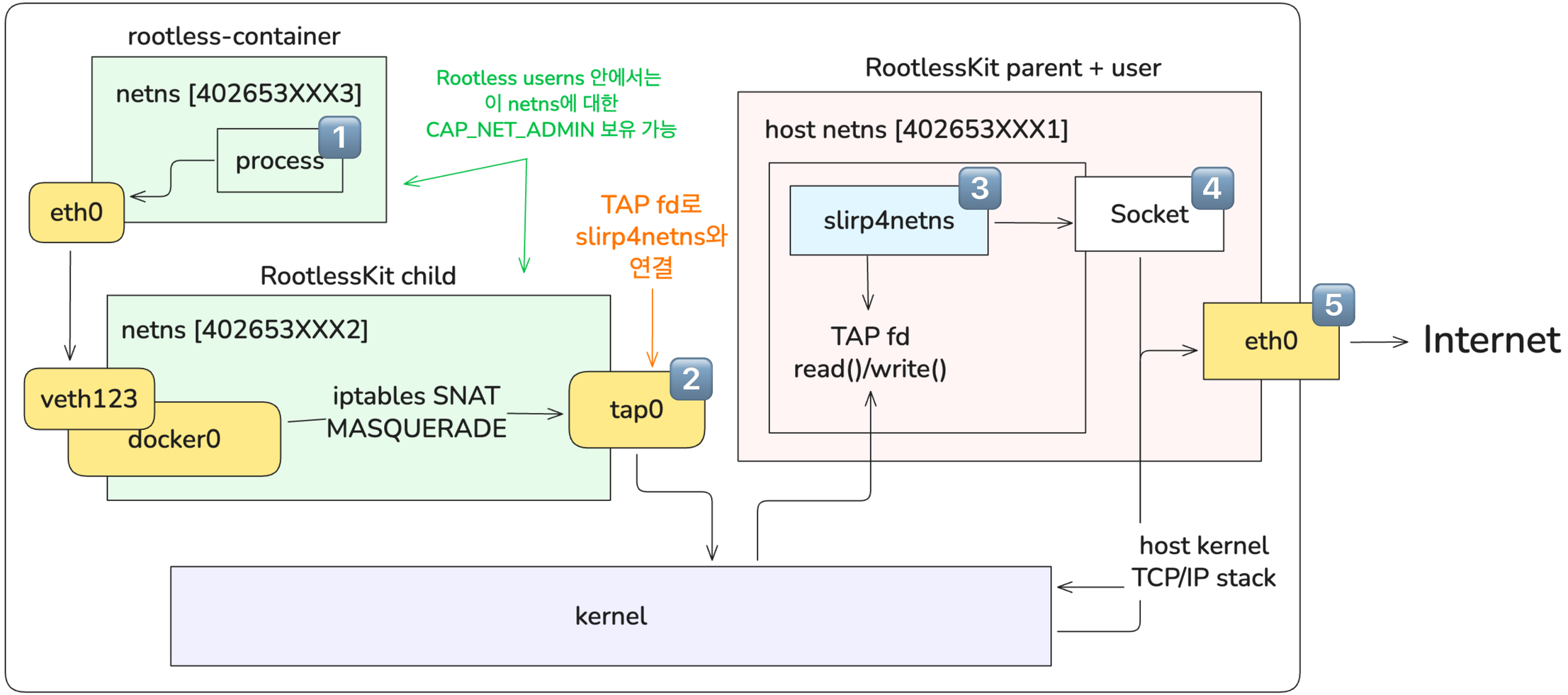
- Almost Internet with SLiRP and PPP (1996.04.01)
  - https://www.linuxjournal.com/article/1174
- passt & pasta: Modern unprivileged networking for containers and VMs
  - https://youtu.be/QMUEtEt1i3I?si=waAxcFItECxL6vc9

수십년이 지난 뒤에도 QEMU VM (kubevirt 등) / Rootless Container에 활용되었음.

- 컨테이너 환경 용도로 개발된 것이 아까 본 slirp4netns 이다.
- 그러나!! 성능 목적으로 개발된게 아님. (즉, Rootful Container에 비해 성능 병목 지점이 있음)

Rootless Container의 네트워크 구성

slirp4netns 기반의 Outbound 통신 (container ➡️ Internet)



Rootless Container의 네트워크 구성

slirp4netns 기반의 Outbound 통신 (container ➡️ Internet)

Rootful Container 환경처럼 host의 netns에 veth 생성 불가

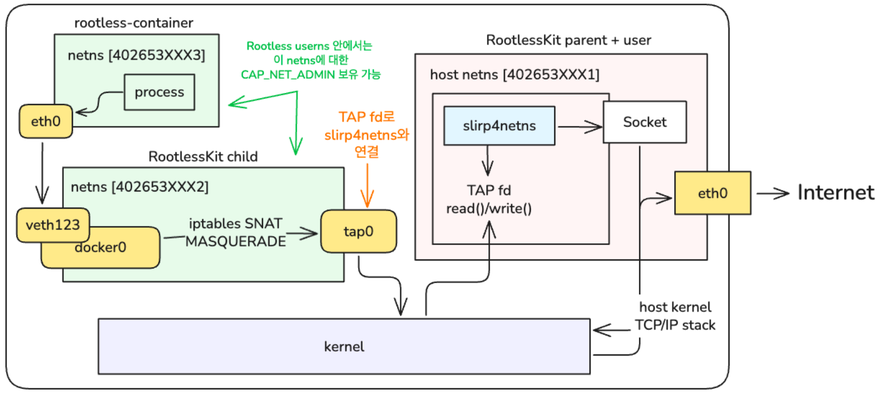
- non-root 유저는 host netns에 대한 CAP_NET_ADMIN 권한이 없기 때문.
- 이를 우회하기 위해 RootlessKit이 별도의 내부 netns를 생성. (RootlessKit Child)

Rootless Child netns에서, 컨테이너 네트워크와 slirp4netns 연결 용도의 tap 인터페이스가 존재.

- tap 인터페이스는 이 Child netns에 속하기에, host netns의 권한이 필요하지 않다.

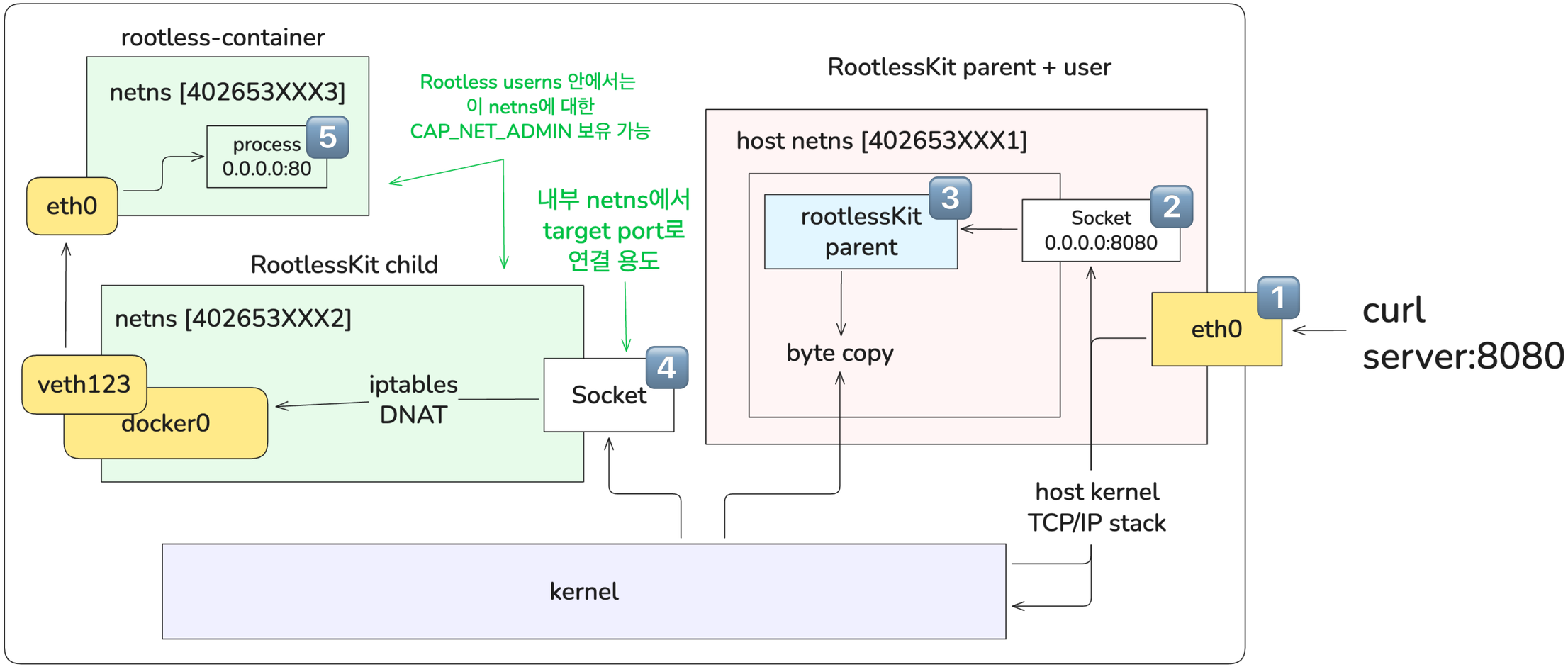
host의 netns에는 slirp4netns와 RootlessKit Parent가 존재한다.

- slirp4netns는 위의 tap 인터페이스의 패킷을 file descriptor 기반으로 인식.
- 이후, host netns 상의 소켓 통신으로 Outbound 통신이 이뤄짐.



Rootless Container의 네트워크 구성

RootlessKit port-forwarder 기반의 Inbound 통신 (container port-forward)



Rootless Container의 네트워크 구성

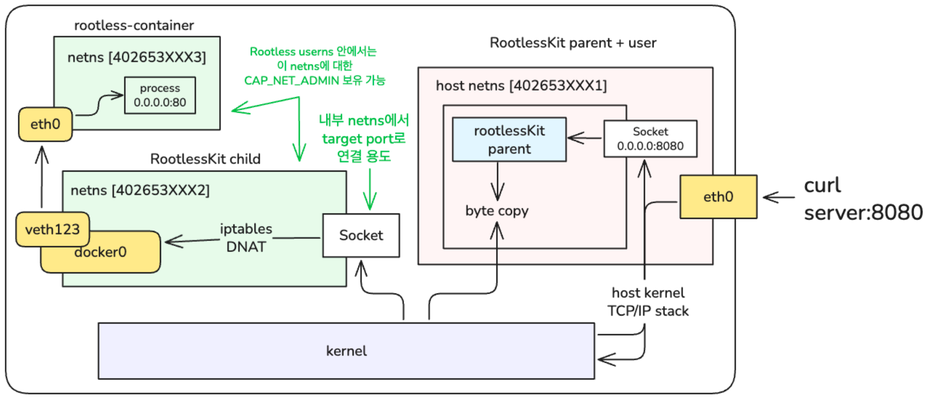
RootlessKit port-forwarder 기반의 Inbound 통신 (container port-forward)

Rootless Container 환경에서는 host netns의 포트를 컨테이너 netns에 직접 연결 불가.

- non-root 유저는 host netns에 대한 CAP_NET_ADMIN 권한이 없기 때문.
- 이를 우회하기 위해 RootlessKit가 host netns와 Child netns 사이의 통신을 중계함.

RootlessKit Parent는 RootlessKit Child 측에 내부 target port로 연결을 요청함.

- 외부 클라이언트가 server:8080으로 접근하면, host netns의 RootlessKit Listener Socket으로 연결됨.
- RootlessKit Child는 내부 netns에서 target port로 연결하기 위한 소켓을 생성.
- 이후 Parent는 외부 client 소켓과 child netns 소켓 사이의 데이터를 byte copy 방식으로 중계한다.



Rootless 환경의 네트워크는 중간 매개체가 필요.

구현체마다 조금씩 다르지만, 결국 병목의 우려가 존재함.

- non-root 유저가 사용하는 host netns는 변경하지 않고, 별도 경유를 위한 netns 만들어 통신을 처리한다.
- slirp4netns의 성능 문제를 해결하려는 구현체도 보이고 있음. (pasta, bypass4netns)
- 단, 결국 host network 조작이 불가능하여, 우회하기 위해 OS 기능을 최대한 응용하는 방식이다.

```
# /home/docker/bin/dockerd-rootless.sh
Guide to choose the network driver and the port driver:

Network driver   | Port driver      | Net throughput | Port throughput | Src IP | No SUID | Note
-----------------|------------------|----------------|-----------------|--------|---------|-----------------------------------------
gvisor-tap-vsock | builtin          | Slow           | Fast ✅         | ✅ (*) | ✅      | Default when slirp4netns is not installed
slirp4netns      | builtin          | Slow           | Fast ✅         | ✅ (*) | ✅      | Default when slirp4netns is installed
vpnkit           | builtin          | Slow           | Fast ✅         | ✅ (*) | ✅      | Legacy
gvisor-tap-vsock | gvisor-tap-vsock | Slow           | Slow            | ❌     | ✅      | Not recommended. Use `builtin` port driver.
slirp4netns      | slirp4netns      | Slow           | Slow            | ✅     | ✅      |
pasta            | implicit         | Slow           | Fast ✅         | ✅     | ✅      | Experimental; Needs recent version of pasta
lxc-user-nic     | builtin          | Fast ✅        | Fast ✅         | ✅ (*) | ❌      | Experimental
(bypass4netns)   | (bypass4netns)   | Fast ✅        | Fast ✅         | ✅     | ✅      | (Not integrated to RootlessKit)
```

정리 및 결론

Rootless Container는 제한된 환경 속에서도 컨테이너 사용할 수 있도록 만들어졌음.

- user_namespace 사용으로 non-root 유저 레벨에서 컨테이너내 root 구성이 가능함.

네트워크는 Rootless Container에서 제일 까다로운 영역이다.

- 기존 host 레벨 netns에서 다루던 영역을 별도의 netns를 통해 우회 처리한다.
- 그러다보니 대부분의 Rootless 네트워크 구현체는 성능에 병목 지점이 드러남.

현실적으로 Production 적용은 아직까지 어려운 부분이 많다.

- 위의 구조로 인해 Rootful Container보다 확인할 지점이 더 늘어난다.
- 제한된 환경 속에서 개발/테스트 용도로 제공하기에는 나쁘지 않겠음.

그렇지만 구현을 시도하는 것 자체가 좋은 방향성으로 보임.

- 시간이 지나서 좀더 나은 구조가 가능해지거나, 혹은 생각지도 못한 활용처가 나올 수도 있을 것임.

End Of Document